Esmaeel Mahfoud — Tasks (Backend Attempts: timing, answers, scoring, results)

Purpose

You build what happens when a user **takes** an interview: starting an attempt, the timer, accepting answers, finishing, timing out, deterministic scoring, the result page data, and the list of the user's attempts.

Read first: `docs/SPECIFICATION.md` (sections 5, 6), `docs/API-CONTRACT.md` (sections 1, 2, 3, 7, 8), `docs/TEAM-OWNERSHIP.md`.

1. Files you own (only you edit these)

File	What goes in it
`backend/src/attempts/attempts.module.ts`	Attempt model, controller, services
`backend/src/attempts/schemas/attempt.schema.ts`	Attempt fields, question snapshot (incl. private scoring data), answers, score fields, indexes
`backend/src/attempts/attempts.controller.ts`	`POST /attempts`, `GET /attempts`, `GET /attempts/:id`, `POST /attempts/:id/answers`, `POST /attempts/:id/finish`, `GET /attempts/:id/results`; `@ApiTags('attempts')`, `@ApiBearerAuth()`
`backend/src/attempts/attempts.service.ts`	Create from interview (reads `interviews` + `questions`

File	What goes in it
	collections natively), ownership check, lazy timeout, answer acceptance, finish, numeric id helper (`counters` key `attempts`)
`backend/src/attempts/scoring.service.ts`	Pure functions: score MC / short answer / code output, percentage, readiness, explanation text
`backend/src/attempts/attempt-serializer.ts`	Build safe `Attempt` / `AttemptQuestion` (no answer keys) and the results response
`backend/src/attempts/dto/create-attempt.dto.ts`	`interviewId`
`backend/src/attempts/dto/attempt-filter.dto.ts`	`status` , `interviewId`
`backend/src/attempts/dto/submit-answer.dto.ts`	`questionId` + answer union by `type`
`backend/src/attempts/dto/attempt-response.dto.ts`	Swagger shapes for Attempt, list item, answer receipt, results
`backend/test/scoring.spec.ts`	Unit tests for scoring (no DB)
`backend/test/attempts.e2e-spec.ts`	Module tests with your own fake guard and raw interview/question documents
`backend/test/helpers/attempts-fake-auth.guard.ts`	Your own copy of a fake guard
`README.md` → section *Attempts, timing and scoring*	Only this heading
`backend/src/app.module.ts`	**Only** add the `AttemptsModule` import line (coordination §3)

2. Files you must NOT edit

- `backend/src/questions/`, `interviews/` (Shiam), `auth/`, `users/`, `main.ts` (Badr), all of `frontend/`.
- You **read** `interviews` and `questions` (contract §8 fields only) — never write to them.
- Never execute code: `code_output` is text comparison only.

3. Implementation order (no one to wait for)

1. **Scoring first** (`scoring.service.ts` + `scoring.spec.ts`). It has no dependencies at all. Cover: MC right/wrong, short answer 0 / half / full with case differences, code output with `\r\n` and extra outer spaces, unanswered, percentage rounding, readiness at 59.9 / 60 / 84.9 / 85.
2. Attempt schema + snapshot design. Decide which question fields are copied (contract §8 lists what you may read).
3. `POST /attempts`: in tests, insert raw `interviews` and `questions` documents with the contract shape.
4. `GET /attempts/:id` with the lazy-timeout step, and the safe serializer.
5. `POST /attempts/:id/answers` with every rule and error code from contract §7.
6. `POST /attempts/:id/finish`, `GET /attempts/:id/results`, `GET /attempts`.
7. Swagger: all endpoints, request bodies (answer union), and every error response.

4. Things to think about

- Two answer requests for the same question arrive at the same time. How do you make sure only one is accepted? (Think about doing the check and the write in **one** atomic database update.)
- Lazy timeout: which endpoints must apply it before doing their work?
- Why must the snapshot store private scoring data even though it is never sent while in progress?
- If the interview is deleted while someone is in the middle of an attempt, what still works and why?

- Profile stats (Badr) read `attempts.status`, `percentage`, `answers` — make sure those names match contract §8 exactly.

5. Completion checklist

- [ ] Attempt can only be read/answered/finished/resulted by its owner (403 otherwise).
- [ ] No answer keys in any in-progress response.
- [ ] Deadline = start + `timeLimitSeconds`; answers accepted until +5 s; later → `ANSWER_LATE` and status `timed_out`.
- [ ] Duplicate, wrong-question, wrong-type and invalid-choice answers rejected with the right codes.
- [ ] Ended attempts are immutable (`ATTEMPT_ENDED`).
- [ ] Results locked until ended (`ATTEMPT_NOT_ENDED`); then show submitted vs expected, points and explanation for each question.
- [ ] Score math and readiness boundaries match the spec exactly.
- [ ] Editing a question after an attempt doesn't change that attempt's results (snapshot).
- [ ] Interviews with 1 question work.
- [ ] Swagger complete.